UNIVERSITAT POLITÈCNICA DE VALÈNCIA

DESARROLLO DE SOFTWARE

EMBOTELLADO Y EMPAQUETADO DE BOTELLAS DE JARABE

GRUPO 2:
Andreu Llorens Martínez
Aarón Poveda Joao
Luisa Fernanda Arango Torres
Kyrylo Korolov
Joan Mallasén Rodríguez

PROYECTO 2:
DISEÑO DE UNA SOLUCIÓN DE INTEGRACIÓN EN EL ÁMBITO DE UNA FÁBRICA

ÍNDICE

1. Introducción y Objetivos 3
1.1. Contextualización del proceso sin automatizar del llenado y empaquetado de Dalsy. 3
1.2. Contextualización del proceso automatizado del llenado y empaquetado de Dalsy. 5
2. Explicación general de los escenarios de integración elegidos para la comunicación 7
2.1. Primer escenario 7
2.1.1. Identificación de Participantes 7
2.1.2. Descripción del Proceso de Integración 8
2.2. Segundo escenario 8
2.2.1. Identificación de Participantes 8
2.2.2. Descripción del Proceso de Integración 9
3. Descripción general del funcionamiento del sistema. 10
3.1. Funcionamiento global de la celda de producción 10
3.2. Gestión de producción basada en base de datos 10
3.3. Seguimiento del proceso de fabricación 10
3.4. Control de condiciones del sistema (temperatura) 11
3.5. Coordinación final de empaquetado 11
4. Arquitectura de datos (Base de datos – GDI) 12
Tabla Lote 12
Tabla Botellas 12
4.2. Lógica de funcionamiento 13
5. Implementación en RoboDK (simulación industrial) 14
5.1. Lógica exclusiva de simulación RoboDK 15
5.2. Comunicación MQTT en RoboDK (Python) 18
5.3. Integración con base de datos en RoboDK 20
6. Implementación en ESP32-S3 (Arduino / C++) 21
6.1. Adquisición de temperatura 21
6.1.1. Explicación de variables 21
6.1.2. Hilos de ejecución y manejador de interrupciones 24

Manejador de interrupciones (ISR: void IRAM_ATTR isr()) 24
Medición de temperatura (Medir_temperatura_NTC_TASK) 25
Tarea del productor (put_item_task) 27
Tarea del consumidor (get_item_task) 27
Listar contenido del buffer (Listar_Buffer_task) 28
Visualización de la LCD (Mostrar_LCD_task) 28
6.1.3 Funciones de comunicación MQTT 28
Suscripción al topic MQTT y Recibir mensajes 28
Enviar Mensajes MQTT 29
Esquema de comunicación 30
6.1.4 Esquema completo del circuito de medición de temperatura: 31
6.2. Sensor de Detección botellas 32
6.2.1. Variables Globales 32
6.2.2. Funciones de comunicación 33
Suscripción al topic 33
Lectura de mensajes del topic 33
Enviar Mensaje Para arrancar el programa: 33
6.2.3 Esquema de comunicación 34
7. Puesta en marcha 36
7.1 Puesta en marcha RoboDK 36
7.1.1 Puesta en marcha Robodk con BD 36
7.1.2 Puesta en marcha Robodk sin BD 37
7.2 Puesta en Marcha Arduino y MQTT 37
8. Anexo de Imágenes 40

1. Introducción y Objetivos

Este proyecto presenta el diseño y prototipado de una solución de integración para una celda automatizada de llenado y empaquetado de botellas de Dalsy. El objetivo principal es coordinar dispositivos embebidos y robóticos para optimizar el flujo de la línea de producción.

Para realizar esta coordinación de dispositivos hemos utilizado:

- Software para programar los diferentes dispositivos
- ESP32-S3 para la parte hardware
- MQTT para la comunicación e intercambio de información

1.1. Contextualización del proceso sin automatizar del llenado y empaquetado de Dalsy.

Para el embotellamiento y empaquetamiento del jarabe Dalsy, se realizaban las siguientes tareas sin automatizar:

a. Un operario tiene que recoger una caja con botellas vacías del medicamento.

b. Este operario se dirige a la zona de rellenado y debe ir rellenando las botellas vacías con el contenido del tanque y colocar el tapón. Una vez la botella se llena y con el tapón puesto, el operario tiene que colocarlas en la caja.
Mientras se da este proceso, el operario tiene que colocar y escanear el código de barras del lote para identificarlo. Si se diera el caso de haber un problema con el contenido del tanque, es decir, que haya un derramamiento del lote u otro problema que puede afectar a la seguridad. El operario tendrá la capacidad de avisar que dicho lote se debe descartar.

c. Una vez la caja está llena, otro operario debe recoger la caja con las botellas perfectamente colocadas y trasladarla a la zona de empaquetado. El operario encargado del llenado repetirá el ciclo de recoger una caja de botellas vacías, rellenarlas y así sucesivamente.

d. Cuando el operario encargado del empaquetado llega a dicha zona, deberá cerrar la caja, ensamblarla y colocarla en el palé para una futura distribución.

Para realizar todas las tareas previamente mencionadas, los operarios deben pasar un tiempo de esterilización obligatorio, debido a que necesitan ir protegidos ante los componentes químicos y también su equipo para evitar que el

jarabe salga contaminado o con defectos que suponen pérdidas y una infracción a la salud.

Además, en el caso donde un tanque se quede sin jarabe, los operarios se deben encargar de reponer el contenido del tanque. Ya sea dando un aviso o activando los mecanismos necesarios para que el tanque vuelva a contener el jarabe.

Veamos dicho proceso siguiendo un diagrama de flujos del proceso sin automatizar:

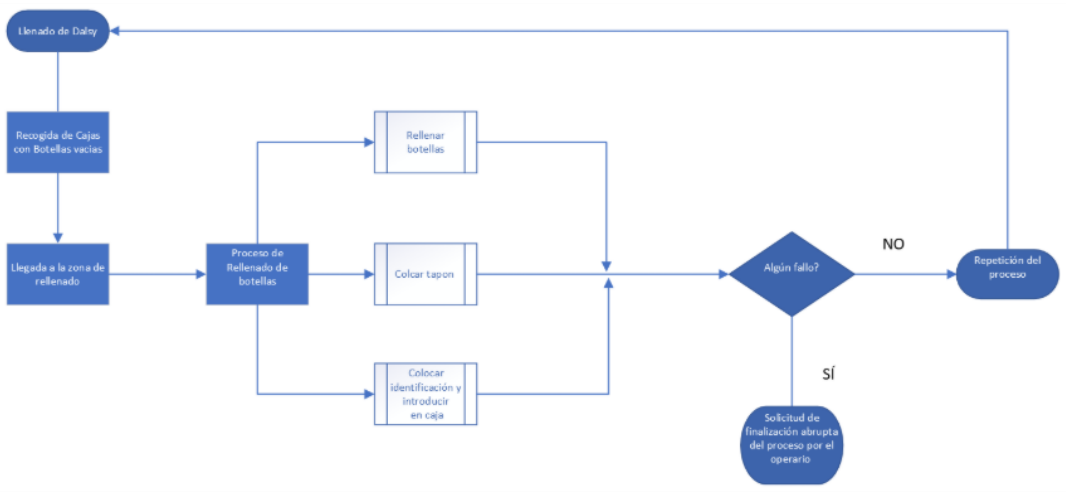

Ilustración 1. Diagrama de Flujo Llenado de Botellas Manual

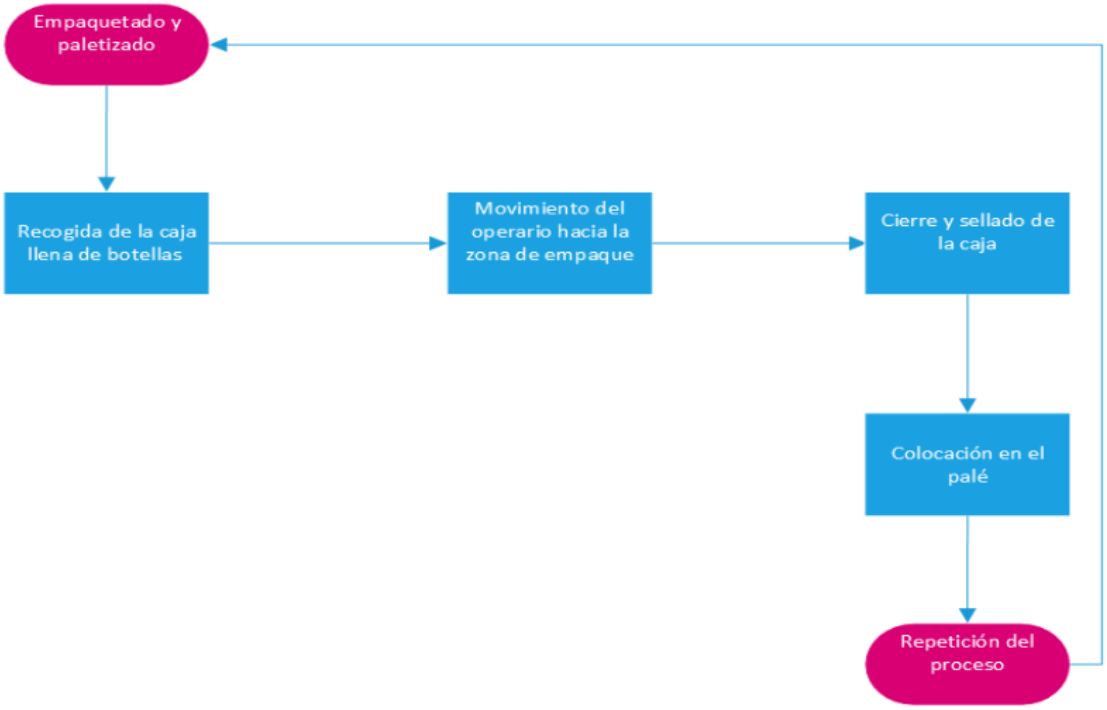

Ilustración 2. Diagrama de Flujo Empaquetado y Paletizado Manual

Como se puede observar el proceso, es altamente repetitivo y se requiere un número elevado de operarios para que la producción del medicamento resulte viable. Además, al manipular compuestos químicos, es imprescindible cumplir normativas muy estrictas con el fin de preservar la calidad del producto, así como la seguridad del operario y consumidor final.

1.2. Contextualización del proceso automatizado del llenado y empaquetado de Dalsy.

Se ha optado por implementar una solución automatizada basada en tres robots colaborativos UR3e, los cuales sustituyen tareas repetitivas previamente realizadas por operarios. Cada robot colaborativo se encarga de una zona específica del proceso.

En primer lugar, se encuentra el **proceso principal de manipulación de botellas**. Las botellas llegan vacías y sin tapón, a través de una cinta transportadora. A medida que avanzan por la línea, las botellas son rellenadas, posteriormente se les coloca el tapón correspondiente y, una vez finalizado este proceso, un robot UR se encarga de recogerlas y depositarlas dentro de una caja situada sobre una mesa giratoria.

El segundo proceso corresponde a la **preparación de las cajas**. Las cajas llegan completamente aplanadas y son manipuladas por un robot UR, que las recoge, les coloca la cinta adhesiva en la parte inferior para cerrarlas y las posiciona sobre la mesa giratoria. Esta mesa permite colocar la caja en la orientación adecuada para el siguiente paso del proceso.

Por último, el tercer proceso se encarga del **cierre y salida de las cajas**. Una vez se haya colocado la botella en la caja, la mesa giratoria vuelve a girar hasta situarla en la posición correcta. Otro robot recoge la caja, la cierra por la parte superior y la deposita sobre la cinta transportadora de salida, donde queda lista para su transporte o almacenamiento.

En conjunto, la celda automatiza todo el proceso desde la llegada de las botellas vacías hasta la salida de las cajas completamente cerradas, integrando robots, cintas transportadoras y una mesa giratoria para garantizar un flujo de trabajo continuo y eficiente.

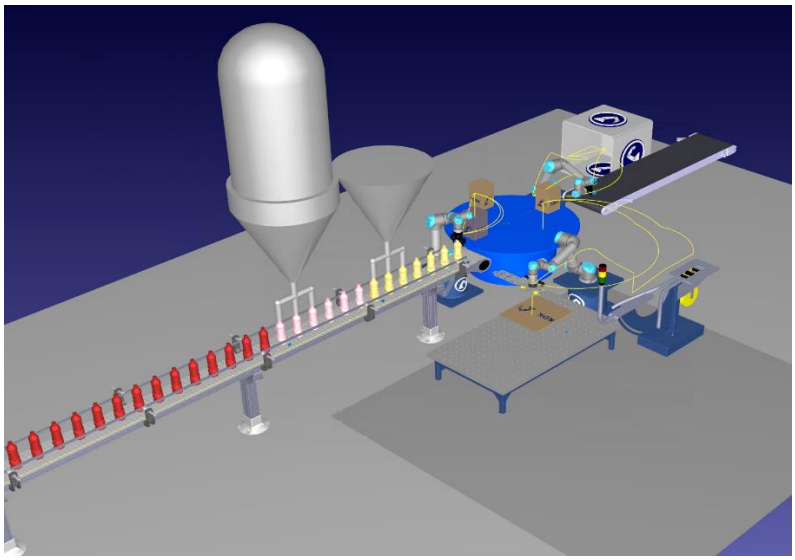

Ilustración 3. Celda Llenado y Empaquetado de Botellas Robotizada

2. Explicación general de los escenarios de integración elegidos para la comunicación

En este proyecto se han definido dos escenarios de integración centrados en el proceso de manipulación de botellas. Ambos escenarios se basan en una arquitectura distribuida en la que los distintos componentes intercambian información mediante el protocolo de conexión MQTT.

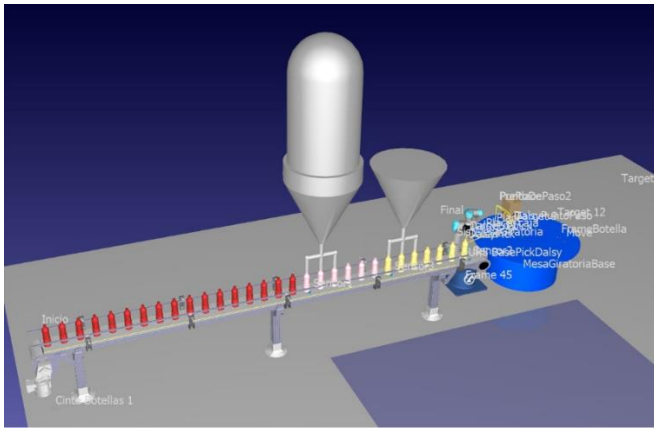

Ilustración 4. Escenarios de Comunicación de la Celda Robotizada

2.1. Primer escenario

2.1.1. Identificación de Participantes

En el escenario de integración propuesto intervienen los siguientes componentes:

- **ESP32-S3**: actúa como nodo de control embebido. Su función principal es la monitorización continua de la temperatura del tanque de jarabe de Dalsy mediante un sensor NTC y la gestión de las comunicaciones mediante MQTT.
- **Sensor NTC**: sensor de temperatura resistivo utilizado para medir de forma continua la temperatura del líquido en el tanque de almacenamiento.

- **Cinta transportadora**: sistema encargado del desplazamiento de las botellas a lo largo de la celda de producción hasta la estación de llenado.
- **Máquina de llenado de botellas (simulada en RoboDK)**: estación robótica encargada del proceso de dosificación del producto en las botellas, cuyo funcionamiento depende de las condiciones del sistema de control.

2.1.2. Descripción del Proceso de Integración

El proceso de integración se basa en una arquitectura distribuida en la que la ESP32-S3 actúa como sistema de monitorización y control, mientras que la simulación en RoboDK gestiona la automatización del proceso robótico.

El sensor NTC realiza una lectura continua de la temperatura del tanque de jarabe de Dalsy. Estos valores son adquiridos por la ESP32-S3, que supervisa constantemente si la temperatura se mantiene dentro de un rango operativo definido.

En caso de que la temperatura salga de dicho rango, la ESP32-S3 envía un mensaje de alerta mediante el protocolo MQTT indicando una condición de fallo o fuera de especificación. Este mensaje es recibido por la simulación en RoboDK a través de scripts en Python (mqttListener y robotController), los cuales actúan como intermediarios de comunicación.

Cuando se detecta esta condición, el sistema robótico detiene el proceso de llenado de botellas para evitar errores en la producción. El proceso permanece detenido hasta que la ESP32-S3 detecta nuevamente que la temperatura ha regresado a los valores correctos y envía una actualización vía MQTT autorizando la reanudación de la operación.

2.2. Segundo escenario

2.2.1. Identificación de Participantes

- **ESP32-S3**: actúa como nodo de control embebido. Su función principal es la recepción y procesamiento de mensajes mediante MQTT, así como la coordinación de las acciones del sistema en función del estado de la celda.

- **Sensor simulado en RoboDK**: elemento virtual integrado en el entorno de simulación que detecta la presencia o ausencia de botellas en la cinta transportadora, generando eventos de control para el sistema.

- **Robot UR3e**: sistema robótico encargado de ejecutar las tareas de manipulación, concretamente la retirada y empaquetado de botellas dentro de la celda automatizada.

2.2.2. Descripción del Proceso de Integración

En este escenario, el sensor se encuentra completamente simulado dentro de RoboDK y su función consiste en detectar la presencia o ausencia de botellas en la cinta transportadora. Cuando se detecta una botella en una posición determinada, se genera un evento dentro de la simulación que provoca la detención de la cinta transportadora.

Este evento es enviado mediante el protocolo MQTT hacia la ESP32-S3, que actúa como nodo intermedio de procesamiento. La ESP32-S3 analiza el mensaje recibido y determina la acción correspondiente en función del estado del sistema.

Una vez procesada la información, la ESP32-S3 envía un nuevo mensaje MQTT al robot UR indicando la acción a realizar. En este caso, el robot debe ejecutar la tarea de retirada de la botella de la cinta transportadora y su posterior empaquetado.

De esta forma, se establece una comunicación continua entre la simulación en RoboDK y el sistema embebido, permitiendo coordinar el comportamiento del sistema robótico en función de eventos detectados en tiempo real dentro del entorno virtual.

3. Descripción general del funcionamiento del sistema.

3.1. Funcionamiento global de la celda de producción

El sistema implementa una simulación de una celda de producción automatizada para la fabricación de botellas de Dalsy, compuesta por una cinta transportadora, una estación de llenado, una estación de taponado y un robot industrial UR3e encargado del empaquetado final.

El comportamiento global del sistema está guiado por la disponibilidad de lotes de producción, la cual se gestiona mediante una base de datos. Cuando existen lotes en estado “pendiente” o “en proceso”, la celda se activa y comienza la fabricación de botellas. En caso contrario, la producción se detiene automáticamente.

3.2. Gestión de producción basada en base de datos

El sistema utiliza una base de datos relacional para gestionar el estado de la producción. En función de la existencia de lotes activos, el sistema inicia o detiene la fabricación.

Durante el proceso, se van generando registros en la tabla de botellas, donde cada unidad producida se asocia a un lote y se actualiza progresivamente su estado a lo largo de la línea de producción.

3.3. Seguimiento del proceso de fabricación

El avance de las botellas a lo largo de la cinta transportadora es supervisado mediante tres sensores digitales distribuidos en las distintas estaciones del sistema.

Cada sensor permite actualizar el estado de la botella en la base de datos:

- Sensor de entrada a llenado → botella pasa a estado “llenada”
- Sensor de entrada a taponado → botella pasa a estado “tapada”
- Sensor de entrada a zona de empaquetado → botella pasa a estado “lista para empaquetado”

Este sistema permite una trazabilidad completa del proceso de fabricación.

3.4. Control de condiciones del sistema (temperatura)

De forma paralela al proceso de producción, un sensor de temperatura monitoriza el estado del tanque de almacenamiento del jarabe.

En caso de detectarse una temperatura fuera del rango operativo definido, la ESP32-S3 envía un mensaje mediante MQTT al sistema de control, provocando la detención inmediata del proceso de fabricación en RoboDK.

El sistema permanece en estado de espera hasta que la ESP32-S3 detecta que la temperatura vuelve a ser correcta, momento en el cual se reanuda la producción.

3.5. Coordinación final de empaquetado

En la fase final del proceso, un sensor ubicado al final de la cinta transportadora detecta continuamente la presencia de botellas en la zona de empaquetado.

La ESP32-S3 recibe esta información y determina si existe alguna botella pendiente de procesamiento final. En caso afirmativo, envía una señal mediante MQTT al robot UR3e para que proceda a la retirada de la botella y su empaquetado.

4. Arquitectura de datos (Base de datos – GDI)

4.1. Diseño del modelo de datos

La base de datos del sistema tiene como objetivo principal gestionar el estado de la producción de la celda automatizada, permitiendo el seguimiento de lotes y botellas a lo largo del proceso de fabricación.

El modelo de datos está compuesto por dos tablas principales:

Tabla Lote

Esta tabla representa cada ciclo de producción del sistema.

```
CREATE TABLE lote (
id SERIAL PRIMARY KEY,
estado VARCHAR(20) NOT NULL CHECK (
estado IN ('pendiente', 'proceso', 'terminado')
    )
);
```

El campo estado permite controlar el estado global del lote de producción, pudiendo encontrarse en los estados:

- **pendiente**: lote creado pero aún no en ejecución
- **proceso**: lote actualmente en producción
- **terminado**: lote finalizado

Tabla Botellas

Esta tabla representa las unidades producidas dentro de cada lote.

```
CREATE TABLE botellas (
  codigo SERIAL PRIMARY KEY,
  llenada BOOLEAN DEFAULT FALSE,
  taponada BOOLEAN DEFAULT FALSE,
  empaquetada BOOLEAN DEFAULT FALSE,
  lote_id INTEGER NOT NULL,

  CONSTRAINT fk_lote
      FOREIGN KEY (lote_id)
      REFERENCES lote(id)
     ON DELETE CASCADE
);
```

Cada registro de botella está asociado a un lote mediante la clave foránea lote_id, lo que permite mantener la trazabilidad completa del proceso de producción.

Los campos booleanos permiten reflejar el estado de cada botella a lo largo de la línea de producción:

- **llenada**: indica si la botella ha pasado por la estación de llenado
- **taponada**: indica si ha sido cerrada correctamente
- **empaquetada**: indica si ha sido manipulada por el robot UR3e en la fase final

4.2. Lógica de funcionamiento

La base de datos actúa como elemento central de control del sistema de producción.

El funcionamiento general es el siguiente:

1. El sistema consulta la tabla lote para comprobar si existen lotes en estado **pendiente** o **proceso**.
2. Si existen lotes activos, se inicia la simulación de la producción en RoboDK.
3. Si no existen lotes activos, el sistema permanece en estado de espera.

Durante la ejecución del proceso, se generan registros en la tabla botellas, donde cada unidad producida se asocia a un lote concreto.

A medida que las botellas avanzan por la línea de producción, su estado se actualiza progresivamente en la base de datos:

1. al pasar por el sensor de llenado → llenada = TRUE
2. al pasar por el sensor de taponado → taponada = TRUE
3. al ser manipulada por el robot UR3e → empaquetada = TRUE

De esta forma, la base de datos proporciona una trazabilidad completa del proceso productivo, permitiendo conocer en todo momento el estado de cada unidad fabricada.

5. Implementación en RoboDK (simulación industrial)

En este apartado se describe la implementación de la solución dentro del entorno de simulación industrial RoboDK. Para ello, se ha diseñado una celda automatizada compuesta por los siguientes elementos:

- Cinta transportadora
- Botellas simuladas
- Tres sensores de presencia (B0, B1 y B2)
- Depósito de llenado
- Depósito de taponado
- Robot colaborativo UR5e
- Mesa giratoria

El funcionamiento de la simulación es el siguiente:

Las botellas avanzan a través de la cinta transportadora hasta alcanzar el sensor B0. Cuando este sensor detecta un conjunto de tres botellas, dichas unidades cambian su color a amarillo, simulando el proceso de llenado.

Posteriormente, las botellas continúan avanzando hasta llegar al sensor B2. Al detectarse nuevamente tres unidades, estas cambian su color a rosa, representando el proceso de taponado.

Finalmente, las botellas llegan al sensor B1, situado en la zona final de la línea. Cuando una botella es detectada en esta posición, la cinta se detiene y se envía un mensaje mediante MQTT a la ESP32-S3. Tras procesar la información recibida, la ESP32 ordena al robot UR5e ejecutar la operación de retirada y empaquetado de la botella.

La simulación desarrollada en RoboDK mantiene la integración completa con el sistema global descrito en apartados anteriores. De este modo, la generación de nuevas botellas depende de la existencia de lotes activos en la base de datos, mientras que el estado de cada unidad se actualiza automáticamente a medida que avanza por las distintas estaciones del proceso. Del mismo modo, la monitorización de temperatura del depósito continúa realizándose mediante la ESP32-S3, de forma que cualquier condición fuera del rango operativo provoca la detención temporal de la producción hasta que los valores vuelven a la normalidad. Todo ello permite que la simulación reproduzca de forma fiel el comportamiento esperado de una celda industrial automatizada e interconectada.

5.1. Lógica exclusiva de simulación RoboDK

Los códigos que se describen a continuación corresponden a rutinas auxiliares del sistema, las cuales no interactúan directamente con la base de datos ni con el protocolo MQTT o, en caso de hacerlo, únicamente realizan consultas de variables o llamadas a otros programas principales.

Para acceder a dichos programas dentro de RoboDK, es necesario dirigirse al árbol de la estación y abrir la carpeta correspondiente a la estación de trabajo. A continuación, dentro del sistema de coordenadas SisGeneral, se debe seleccionar la carpeta Cinta y Robot.

Una vez realizado este recorrido, se visualizará un sistema de referencia junto con tres carpetas principales, tal y como se muestra en la siguiente imagen.

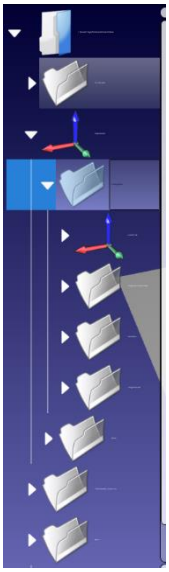

Ilustración 5. Organización en Árbol y Carpetas de la Celda en RoboDK

Primera Carpeta

De esta forma, comenzamos con la primera carpeta, denominada Programas de la Cinta. Dentro de ella, resultan relevantes tres programas principales para el funcionamiento de la simulación.

El primero de ellos es ResetBot, desarrollado en Python. Su ejecución elimina todas las botellas presentes en la simulación y restablece las variables del sistema a sus valores iniciales, permitiendo reiniciar el proceso desde un estado limpio.

El segundo programa es ProgCinta1, desarrollado mediante programación propia de RoboDK. Su función principal consiste en consultar tres variables de control

antes de ejecutar el programa que se explicará a continuación, denominado ProgCintaUno.

Las variables verificadas son las siguientes:

- Detectado1: indica si existe una botella situada al final de la cinta transportadora. En caso de que su valor sea TRUE, la cinta se detiene temporalmente.
- Temperatura: si se detecta una condición de temperatura alta, no se permite continuar la ejecución del proceso.
- LotesPendientes: indica si existen lotes activos o pendientes de producción. Solo en ese caso se ejecuta el siguiente programa.

Más adelante se detallará en qué módulos del sistema se modifica cada una de estas variables.

El tercer y último programa de esta carpeta es ProgCintaUno. Este programa se encarga de realizar las operaciones principales sobre la cinta transportadora, entre las que destacan la creación de nuevas botellas al inicio de la línea, su numeración secuencial y el desplazamiento continuo de las mismas a lo largo de la cinta.

Segunda Carpeta

En la segunda carpeta, denominada Sensores, se encuentran tres programas desarrollados en Python, cada uno asociado al comportamiento de los sensores presentes en la línea de producción.

El primero de ellos es ProgB0. Este programa se encarga de detectar y contabilizar grupos de tres botellas. Una vez identificadas, las botellas son coloreadas de amarillo para simular el proceso de llenado. Posteriormente, se ejecuta el programa LlenarBD, cuya función se explicará en apartados posteriores.

El segundo programa es ProgB1. Este módulo combina funcionalidades propias de RoboDK con comunicación mediante MQTT, por lo que será desarrollado con mayor detalle en ambos apartados correspondientes. Se trata del programa asociado al sensor situado al final de la cinta transportadora. Cuando detecta la presencia de una botella, envía un mensaje a la ESP32-S3 indicando dicha condición; en caso contrario, transmite un mensaje diferente informando de la ausencia de producto. Su funcionamiento dentro de la arquitectura MQTT se explicará más adelante.

El tercer programa es ProgB2. Su funcionamiento es similar al de ProgB0, ya que detecta grupos de tres botellas y las colorea de rosa para simular el proceso de

taponado. Tras ello, ejecuta el programa TaponarBD, encargado de actualizar la información correspondiente en la base de datos, el cual será descrito posteriormente.

Tercera carpeta

En la tercera carpeta, denominada ProgramasUR, se encuentran cinco programas relacionados con el funcionamiento del robot industrial.

El primero de ellos, denominado DalsyPickPlace, no será considerado en esta entrega, ya que ha sido sustituido por una versión más actualizada llamada RobotPickAndPlace, ubicada igualmente dentro de la misma carpeta.

El funcionamiento de RobotPickAndPlace es sencillo. En primer lugar, el programa permanece en espera hasta que la mesa se encuentra en la posición adecuada. No obstante, para esta entrega no se ha tenido en cuenta la lógica asociada a la mesa giratoria, por lo que dicho comportamiento queda simplificado. Además, el programa espera a que la variable SensorMQTT adopte el valor correspondiente, variable que será modificada por los scripts descritos en el apartado dedicado a MQTT.

Una vez cumplidas estas condiciones, el robot inicia la secuencia de movimientos programados entre distintos puntos de trayectoria. Durante dicha secuencia se invocan los programas PickDalsy y PlaceDalsy, también ubicados en esta carpeta, encargados respectivamente de simular la recogida de la botella y su posterior deposición en la zona de destino.

Por último, existe otro programa denominado BorrarBotella, el cual es ejecutado desde PlaceDalsy con el objetivo de eliminar la botella de la simulación una vez ha sido depositada, representando así el final del proceso de empaquetado.

5.2. Comunicación MQTT en RoboDK (Python)

A continuación, se describen los programas implementados en RoboDK para establecer la comunicación mediante el protocolo MQTT con la ESP32-S3.

Para ello, dentro del árbol de la estación se accede a la carpeta MQTT, donde se encuentran los dos programas principales responsables de esta integración: MqttListener y RobotController.

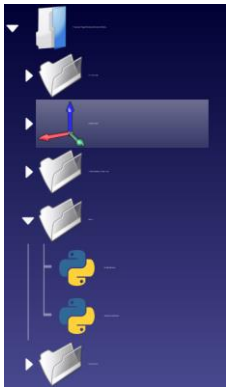

Ilustración 6. Scripts MqttListener y RobotController

Ambos módulos permiten el intercambio de información entre la simulación industrial y el sistema embebido, posibilitando la supervisión de variables del proceso y la coordinación de acciones en tiempo real dentro de la celda automatizada.

El programa MqttListener se encarga de la configuración inicial del sistema MQTT y de gestionar la conexión entre RoboDK y la ESP32-S3. Para ello, se emplean los siguientes parámetros:

- Broker: broker.emqx.io
- Puerto: 1883
- Usuario: giiroB22
- Contraseña: UPV2024
- Topic de recepción: giirob/pr2/station/mesa/commands
- Topic de envío: giirob/pr2/station/mesa/status

Se ha utilizado un broker público con el objetivo de facilitar las pruebas remotas y permitir el desarrollo desde distintos entornos.

El topic commands se utiliza para la recepción de órdenes en RoboDK, mientras que el topic status se emplea para el envío de información del estado del sistema.

Una vez establecida la conexión con el broker MQTT, el sistema publica el mensaje de estado *online* en el topic correspondiente, indicando que la estación está operativa.

El programa RobotController es invocado automáticamente por MqttListener cada vez que se recibe un mensaje en el topic commands. Su función consiste en interpretar dichos mensajes y actualizar las variables internas del sistema.

Cuando se recibe el mensaje Temperatura_alta, la variable Temperatura se establece a 1. En cambio, si se recibe Temperatura_correcta, dicha variable vuelve a 0. Tal y como se ha indicado previamente, mientras esta variable esté activa, el proceso de producción se detiene por motivos de seguridad.

Por otro lado, cuando se recibe SensorMqtt, la variable SensorMQTT se activa (1), mientras que con el mensaje NotSensorMqtt se desactiva (0). Esta variable es utilizada por el robot UR para determinar la presencia de una botella en la zona final de la cinta y ejecutar su retirada y empaquetado.

De este modo, RobotController actúa como intermediario entre la comunicación MQTT y la lógica interna de control de RoboDK.

Por último, el programa ProgB1 se encarga de publicar el estado del sensor situado al final de la cinta transportadora.

Este programa envía mensajes al topic status, indicando si se ha producido detección de botella. En concreto, publica el mensaje sensor_detecta cuando existe colisión entre la botella y el sensor, y sensor_no_detecta cuando no se registra dicha interacción.

De esta forma, se mantiene una comunicación continua del estado final de la línea, permitiendo que la ESP32-S3 y el resto del sistema reaccionen en función de la presencia o ausencia de botellas en la estación final.

5.3. Integración con base de datos en RoboDK

Este apartado describe los programas ubicados en la carpeta **BaseDatos**, responsables de la interacción entre la simulación y la base de datos relacional, permitiendo la gestión de lotes y el seguimiento del estado de las botellas a lo largo del proceso productivo.

Dentro de la primera subcarpeta se encuentran varios scripts principales. El primero de ellos es ConsultarBD, cuyo objetivo es consultar de forma continua el estado de la base de datos para verificar si existen lotes en estado *pendiente* o *en proceso*. Mientras se cumpla esta condición, el sistema activa la variable LotesPendientes, que permite habilitar la producción en la celda.

El script CrearBotellaBD es invocado desde el programa ProgCintaUno y se encarga de insertar nuevas botellas vacías en la base de datos, asociándolas al lote correspondiente.

Por otro lado, LlenarBD, TaponarBD y EmpaquetarBD son scripts encargados de actualizar el estado de cada botella a lo largo de la línea de producción. En concreto, LlenarBD se ejecuta desde ProgB0 para marcar las botellas como llenadas, TaponarBD se ejecuta desde ProgB2 para indicar que han sido taponadas, y EmpaquetarBD se invoca desde PlaceDalsy para registrar su empaquetado final.

El script ResetBD se utiliza para reiniciar completamente el sistema de datos, eliminando todos los registros de lotes y botellas. Este proceso se ejecuta al inicio de la simulación para garantizar un estado limpio de la base de datos.

En la segunda subcarpeta se encuentran dos scripts adicionales. CrearLotesBD se ejecuta una única vez al inicio de la simulación y es el encargado de crear un nuevo lote en estado *pendiente*, iniciando así el ciclo de producción. Por último, TerminarLotesBD monitoriza el número de botellas empaquetadas y, cada 30 unidades completadas, actualiza el estado del lote correspondiente a *terminado*, cerrando así el ciclo productivo.

En conjunto, estos programas permiten la gestión completa del sistema de producción, asegurando la trazabilidad de cada botella y el control del estado de los lotes en todo momento.

6. Implementación en ESP32-S3 (Arduino / C++)

6.1. Adquisición de temperatura

El código ".ino" localizado en el fichero Medicion_temperatura (Medicion_temperatura/ESP32-S3-IoT-Device) es el encargado del proceso de medición de temperatura del tanque. El módulo se basa en una arquitectura concurrente compuesta por seis hilos (tareas) además del hilo principal que permiten la medición de temperatura, almacenamiento en un buffer circular, visualización y control de emergencia ante diversas situaciones, como es el caso, de una parada de emergencia o un dato de temperatura anómalo.

6.1.1. Explicación de variables

En primer lugar, se detallan los tipos de variables y las variables globales utilizadas a lo largo de la ejecución del programa, definidas todas en el fichero "Config.h"

VARIABLES MENSAJE MQTT
#define ENVIAR_MENSAJE_TEMEPRATURA_ALTA **"temperatura_alta"** #define ENVIAR_MENSAJE_TEMPERATURA_CORRECTA **"temperatura_correcta"** #define MENSAJE_ACTIVAR_SETA **"EMERGENCIA"** #define MENSAJE_DESACTIVAR_SETA **"LIBERANDO"**
Las dos primeras constantes representan el estado de la temperatura medida por el sensor que se envían por MQTT, mientras que las dos últimas corresponden al estado de la seta de emergencia. Al activarse la seta, se envía el mensaje "EMERGENCIA" mediante MQTT y se suspenden las tareas de ejecución y la simulación. Al desactivarlo, se envía "LIBERANDO" y se reanudan todas las tareas.

VARIABLES RANDO DEL VALOR ANALOGICO (ADC)
#define RANGO_ANALOGICO_MAX 4095 #define RANGO_ANALOGICO_MIN 0
Define los valores mínimos y máximos que puede tomar la lectura analógica del sensor de temperatura.

VARIABLES PAUSA DE TAREAS
#define TIEMPO_ESPERA_PUT 1000 #define TIEMPO_ESPERA_GET 1000 #define TIEMPO_ESPERA_LISTAR 1500 #define TIEMPO_ESPERA_MEDIR_NTC 1000 #define TIEMPO_PAUSA_DE_TAREAS 1000
Estas constantes establecen los tiempos de espera (en milisegundos) de las distintas tareas. En particular hay que destacar "TIEMPO_PAUSA_DE_TAREAS" que se utiliza para la pausa en bucle de la ejecución cuando se activa la seta de emergencia.

VARIABLES PARA LOS ELEMENTOS CONECTADOS
#define PIN_NTC 19 //pin NTC #define PIN_SDA 4 //pines para la lcd #define PIN_SCL 5 #define PIN_LED_ROJO 20 #define PIN_LED_AZUL 48 #define PIN_BOTON_EMERGENCIA 13 LiquidCrystal_I2C lcd_i2c(0x3F, 16, 2); //Variable de acceso al lcd
Estas variables indican la conexión de los distintos dispositivos a la ESP32-S3. El sensor NTC se emplea para la medición de temperatura, mientras que los pines SDA y SCL se utilizan para la comunicación con una pantalla LCD que muestra los valores de temperatura obtenidos. Los LEDs indican el estado del sistema, por ejemplo, el led azul simboliza que se han activado los sistemas de refrigeración. Y por último la seta de emergencia que permite interrumpir el proceso.

VARIABLES PARA LA TEMPERATURA

const float Vcc = 3.3f;
const float R0 = 10000.f;
const float beta = 3950.0f;
const float T0 = 298.15f; //Temperatura ambiente que suponemos (25ºC)
const float K = 273.15f; //para pasar a Celsius
const float Temperatura_minima_permitida = 20.0f;
const float Temperatura_maxima_permitida = 32.0f;

Estas constantes permiten calcular la temperatura a partir del sensor NTC, además de definir el rango de funcionamiento.

VARIABLES DE CONTROL DE ESTADOS

const unsigned long tiempoAcomodo = 5000; // 5000 milisegundos = 5 segundos extra
unsigned long inicioAcomodo = 0;
bool enAcomodo = false;

Estas variables se utilizan para evitar el envío repetitivo de estados de temperatura, introduciendo un tiempo de estabilización.

ESTADO DEL SENSOR

enum Estado_Temperatura
{
  CORRECTA,
  ALTA,
  DESCONOCIDO
};

Enumeración de los posibles estados asociados a la temperatura medida.

SEMÁFOROS DE ACCESO A VARIABLES GLOBALES
portMUX_TYPE temperature_buffer_access; portMUX_TYPE temperatura_medida_NTC_access; portMUX_TYPE medicion_extraida_buffer_access;
Estos semáforos garantizan el acceso seguro a las variables globales compartidas entre tareas, evitando condiciones de carrera. Todos se inicializan en estado desbloqueado.

VARIABLES GLOBALES

volatile bool Seta_ISR = false;
volatile bool Estado_ISR = false;
volatile float Temperatura_medida_NTC = 0.0f;
volatile float Medicion_extraida_buffer = 0.0f;
Estado_Temperatura Estado_temperatura_actual = DESCONOCIDO;
Buffer_Circ buf;

Son las variables compartidas entre múltiples hilos. Son del tipo “volatile” para garantizar su actualización.

Además, el sistema implementa una cola mediante un buffer circular (Buffer_Circ), donde se almacenan las mediciones de temperatura siguiendo una política FIFO (First In, First Out). Este buffer cuenta con una capacidad máxima de 10 valores de temperatura. Y es la tarea productora quien introduce nuevos valores de temperatura, mientras que la tarea consumidora va vaciando poco a poco el buffer circular.

6.1.2. Hilos de ejecución y manejador de interrupciones

En segundo lugar, el fichero correspondiente a la adquisición de la temperatura presenta una arquitectura concurrente mediante el uso de hilos de ejecución. Estos hilos permiten la adquisición de la temperatura, el procesamiento de esta y la comunicación.

Manejador de interrupciones (ISR: void IRAM_ATTR isr())

Se basa en una rutina de interrupción que solamente está asociada al botón de emergencia.

El funcionamiento de dicha interrupción se basa en lo siguiente:

1. Cuando se detecta la pulsación del botón se ejecuta el manejador que cambia el estado de la variable “Seta_ISR”
2. La variable “Seta_ISR” es una variable global que activa y desactiva el modo de emergencia mandando uno de estos mensajes vía MQTT: “EMERGENCIA” o “LIBERADO”
3. Cuando ha habido un cambio en “Seta_ISR” las tareas del sistema pueden entrar en estado de pausa controlada, deteniendo la ejecución hasta que se le vuelva a dar el botón.

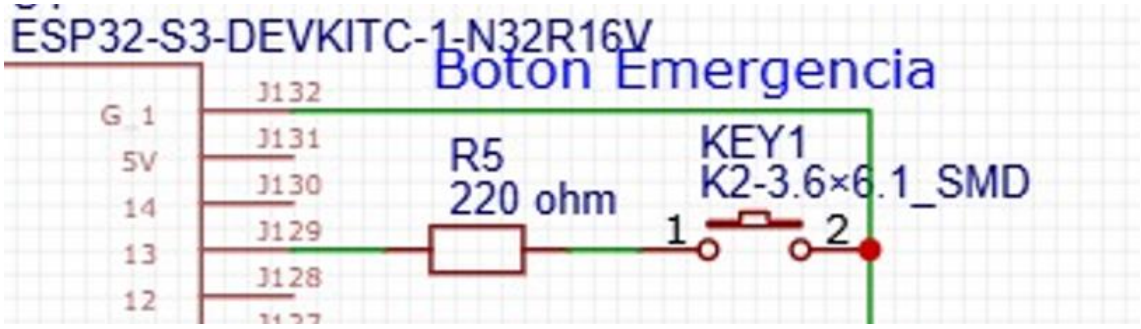

Ilustración 7. Esquema de Conexión ESP32-Seta Emergencia

Medición de temperatura (Medir_temperatura_NTC_TASK)

Este hilo se encarga de realizar la lectura continua del sensor NTC.

Este hilo obtiene el valor de temperatura que devuelve la función Medir_temperatura(). Donde se realizan los siguientes cálculos para obtener el valor exacto de la temperatura del tanque:

1. Obtiene el valor analógico de la lectura del sensor a través de la función "analogRead(PIN_NTC)"
2. Con este valor analógico obtenemos el voltaje que recibe el ESP32 a través de la siguiente expresión matemática:

$$V_{\{out\}} = \frac{\{ADC_{\{valor\}} \cdot V_{\{cc\}}\}}{\{RANGO_{\{ANALOGICO_MAX\}}\}}$$

3. Con dicho valor ya podemos obtener el valor exacto cuánto ha variado el valor de la resistencia NTC

$$R_{\{NTC\}} = \frac{\{V_{\{out\}} \cdot R_0\}}{\{V_{\{cc\}} - V_{\{out\}}\}}$$

4. Y ya con todos los valores que podemos extraer de la NTC ya podemos calcular la temperatura que hay en el circuito. Valor de la expresión en Kelvins.

$$T_{\{K\}} = \frac{\{1\}}{\frac{\left\{ln\left(\frac{\{R_{\{NTC\}}\}}{\{R_0\}}\right)\right\}}{\{\beta\}} + \frac{1}{\{T_0\}}}$$

Para obtener la expresión en Grados centígrados:

$$T_{\{C\}} = T_{\{K\}} - 273.15$$

Con lo que podemos deducir de estas expresiones a mayor temperatura disminuye el valor de la resistencia NTC. Veamos esto con la curva característica de una NTC obtenida de la siguiente expresión:

$$R_{\{NTC\}}(Temperatura) = R_0 e^{\beta(\frac{1}{T_{\{C\}}} - \frac{1}{T_0})}$$

Sustituyendo con los valores de las variables globales utilizados:

$$R_{\{NTC\}}(Temperatura) = 10000\ e^{3950.0(\frac{1}{T} - \frac{1}{298.15})}$$

Veamos la gráfica que obtenemos al darle valores entre 0ºC y 50ºC (273,15 – 323,15 Kelvins):

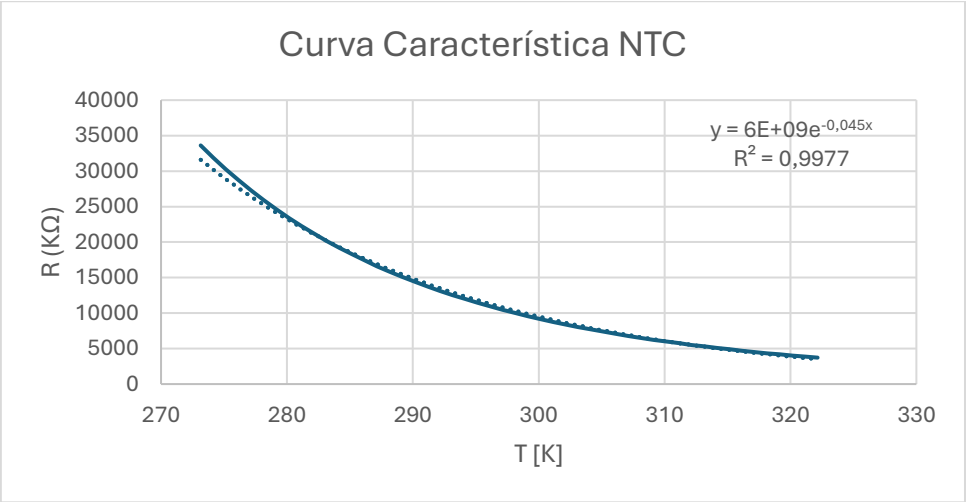

Ilustración 8. Curva Característica NTC

Como se puede ver a mayor temperatura menos resistencia. Por lo tanto, este hilo nos proporciona el valor de temperatura contiene el tanque frecuentemente.

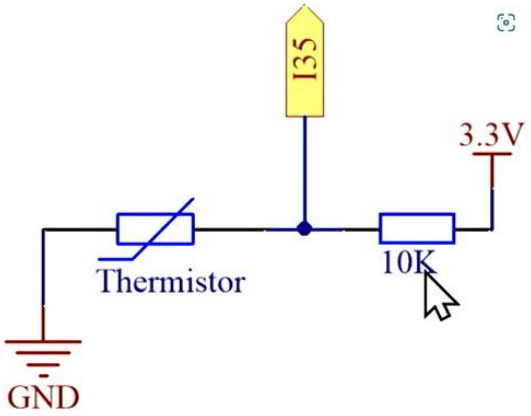

Ilustración 9. Esquema de Conexión NTC

Tarea del productor (put_item_task)

Obtiene de manera segura y protegida, mediante el uso de semáforos para acceder a la sección crítica, la última medida extraída de la tarea de medición de temperatura. La variable que comunica ambas tareas, la de medición de temperatura y la productora, es la "volatile float Temperatura_medida_NTC". Dicho valor se introduce en el buffer circular siguiendo una política FIFO.

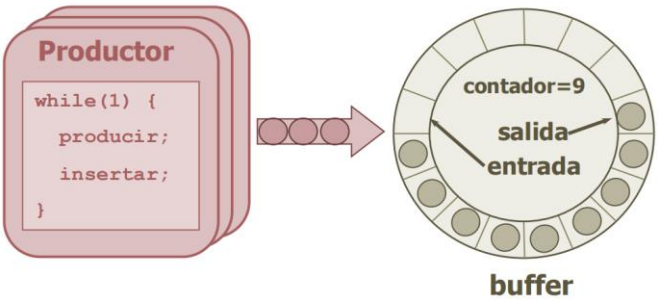

Ilustración 10. Tarea Productora

Tarea del consumidor (get_item_task)

Esta tarea recupera el valor más antiguo que contenga el buffer Circular. Dicho valor extraído es almacenado en una variable global "Medicion_extraida_buffer" que posteriormente dicho valor será enviado para analizar si la temperatura está en los rangos correspondientes para su futuro envío por MQTT y para su visualización mediante el uso de la pantalla LCD. Dicho valor global, siempre se asegura que no se produzca una condición de carrera con el uso de semáforos.

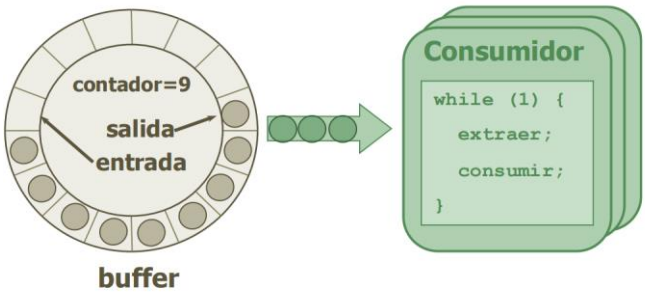

Ilustración 11. Tarea Consumidor

Listar contenido del buffer (Listar_Buffer_task)

Esta tarea accede al contenido del buffer de forma segura. Una vez accede al buffer realiza una copia temporal de los datos de medición de temperatura que se encuentren en el buffer en ese momento. Dichos valores copiados se van mostrando por la consola con cierta frecuencia.

Visualización de la LCD (Mostrar_LCD_task)

Este hilo obtiene el valor extraído del buffer de manera segura, lo copia y dicha copia se visualiza por la pantalla LCD.

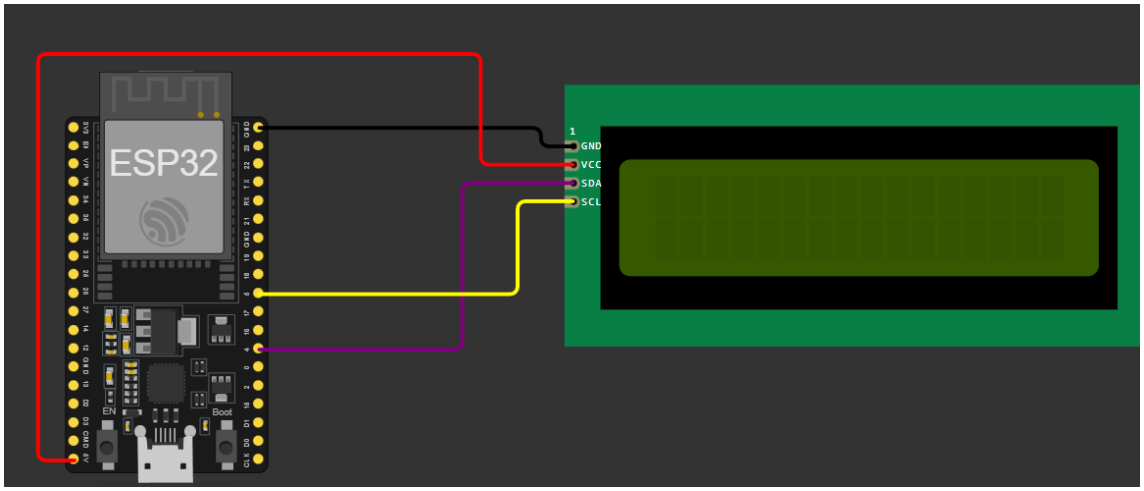

Ilustración 12. Esquema de Conexión ESP32-LCD

Todas las tareas que comparten información mediante variables globales están protegidas por mecanismos de exclusión mutua (SEMÁFOROS DE ACCESO A VARIABLES GLOBALES). Esto se obtiene mediante el uso de las expresiones “portENTER_CRITICAL” y “portEXIT_CRITICAL”

6.1.3 Funciones de comunicación MQTT

Para realizar la conexión MQTT, se implementan un conjunto de funciones encargadas de la suscripción al topic, la recepción de mensajes y el envío de estados del dispositivo

Suscripción al topic MQTT y Recibir mensajes

La ESP32 se suscribe al topic: **"giirob/pr2/station/mesa/status”** durante la inicialización de la conexión MQTT. Aunque el dispositivo reciba los mensajes publicados en el topic, en este escenario de integración no se define ninguna acción específica asociada al contenido de un mensaje recibido. Por lo que los mensajes recibidos no provocan ningún cambio en el comportamiento del escenario.

Enviar Mensajes MQTT

La función enviarMensajePorTopic gestiona el envío de estados del sistema hacia la conexión del servidor MQTT. Esta función enviará los siguientes mensajes

1. Estado del botón de emergencia:

   "EMERGENCIA" o "LIBERADO" dependiendo el estado del botón de emergencia.

2. Estado de la temperatura del tanque:

   Se monitoriza la temperatura del tanque y se compara el valor de la temperatura extraída del sensor con el valor máximo permitido (Temperatura_maxima_permitida = 32ºC).

   - Si la temperatura supera el umbral de 32º, en el canal de MQTT se envía el mensaje "temperatura_alta".
   - Si la temperatura se encuentra en el rango ideal, se envía "temperatura_correcta".

   Estos mensajes se transmiten únicamente cuando se produce un cambio de estado en la temperatura (de correcta a alta, o viceversa) con el objetivo de evitar una saturación del canal MQTT.

3. Acciones en caso de temperatura alta:

   Si se diera el caso de que el programa tiene que enviar el mensaje de "temperatura_alta, en la simulación de roboDK se detienen la cinta transportadora y la extracción de fluidos (las botellas no cambiarían de color, ni avanzarían). Además, se enciende un LED azul que indica la activación de los sistemas de refrigeración para reducir la temperatura (ver esquemático del circuito)

Los mensajes se publican en el topic **"giirob/pr2/station/mesa/commands"**, el cual es leído por RoboDK, permitiendo así la interacción entre el sistema de control de la temperatura y la simulación. Como se ha mencionado anteriormente, cuando en RoboDK le llegue el mensaje de "temperatura_alta", la cinta transportadora se detiene y no se continúa con el proceso de llenado de botellas.

Todo este proceso de análisis y decisión sobre qué mensaje se debe enviar se realiza dentro de la función enviarMensajePorTopic, siempre utilizando el valor de temperatura extraído del buffer. Por este motivo, se debe proteger el acceso a la sección crítica para garantizar el correcto funcionamiento de todos los hilos de ejecución del programa.

Esquema de comunicación

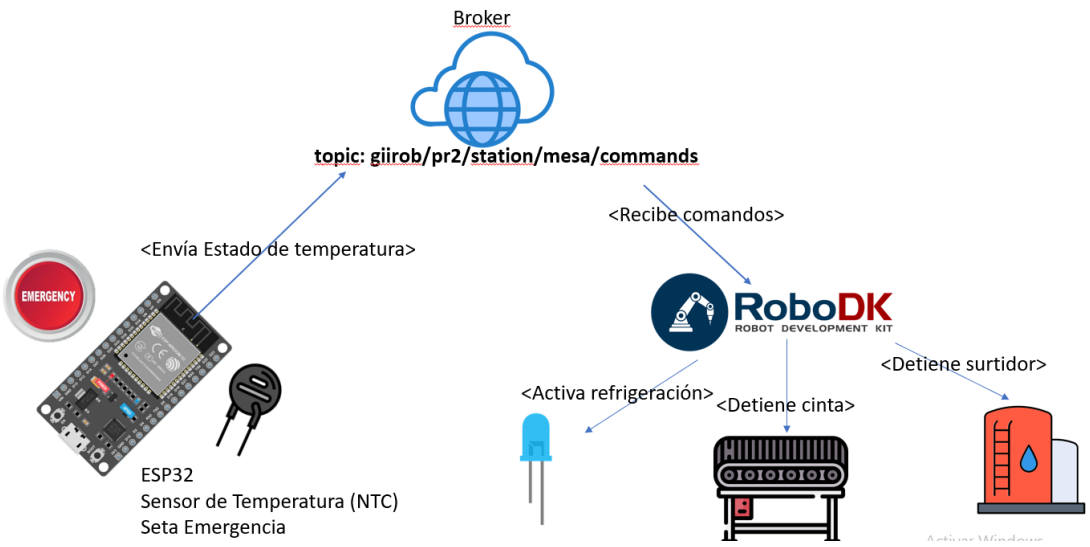

Ilustración 13. Esquema de Comunicación Escenario 1

1. El ESP32 envía al topic “giirob/pr2/station/mesa/commands” el estado de la temperatura
2. RoboDK leerá dicho estado y dependiendo de que ha analizado hará lo siguiente:
    a. Si la temperatura es correcta el proceso continuará con normalidad
    b. Si la temperatura es elevada RoboDK dará la orden de detener la cinta transportadora y el surtidor de líquido. Y activará el sistema de refrigeración
    c. En el caso que se haya pulsado a la seta de emergencia, los hilos de ejecución se detendrán y la simulación de RoboDK se detendrá también o viceversa.

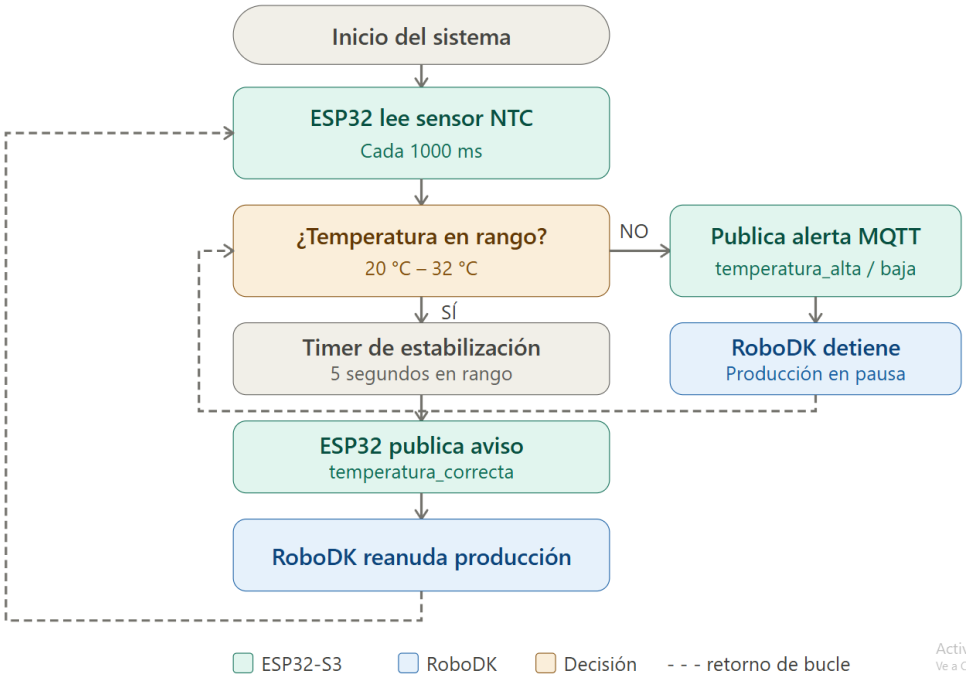

Ilustración 14. Diagrama Flujo Información NTC-ESP32-MQTT

6.1.4 Esquema completo del circuito de medición de temperatura

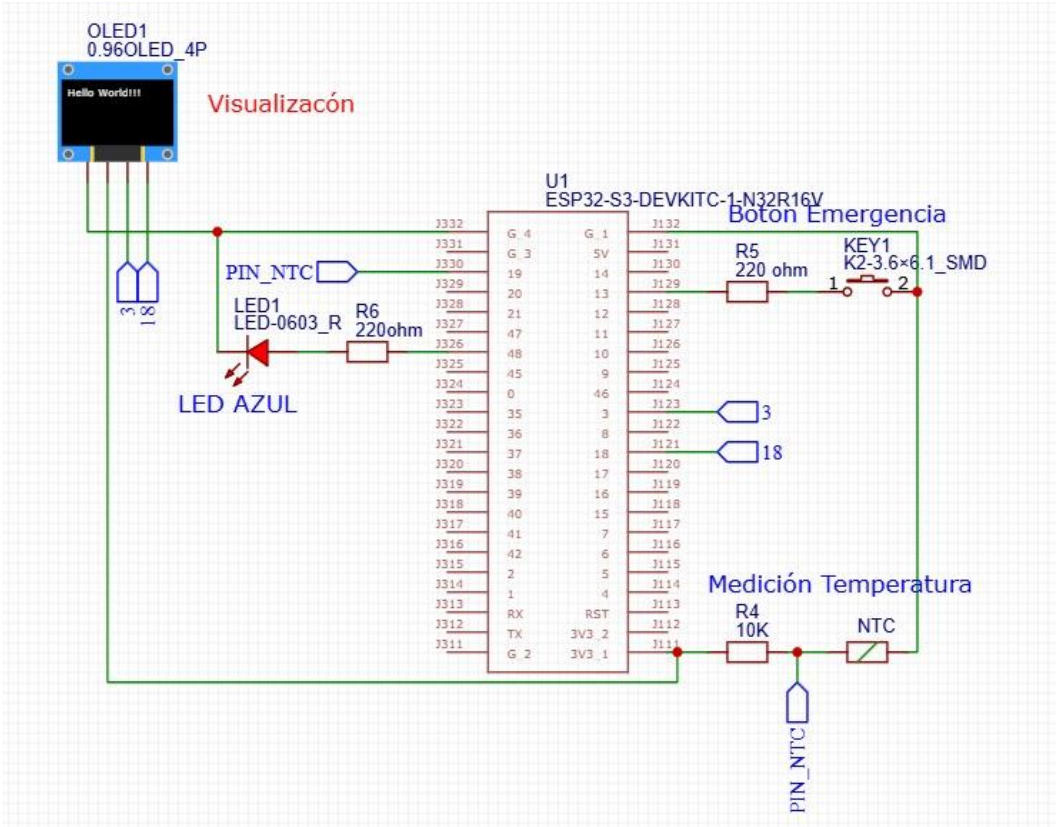

Ilustración 15. Esquema Conexión Circuito de Medición de Temperatura

6.2. Sensor de Detección botellas

El módulo de detección de botellas forma parte del segundo escenario de integración. Este proceso se encarga de simular un sensor implementado en RoboDK, la detección de las botellas cuando lleguen al fin de la cinta.

Este módulo se encuentra en el fichero llamado “Sensor_botella” dentro del fichero “Codigos_ino_mqtt” del repositorio de github.

Este módulo nos permite conectar y simular un sensor capacitivo desde el entorno de RoboDK. La ESP32 recibirá el mensaje que una botella se ha detectado y posteriormente le mandará la señal al robot que ya puede iniciar el programa para la recogida de la botella.

6.2.1. Variables Globales

Las únicas variables globales que intervienen son las siguientes:

MENSAJES MQTT
#define MENSAJE_DETECTADO "sensor_detecta" #define MENSAJE_NO_DETECTADO "sensor_no_detecta" #define ENVIAR_SENYAL "SensorMqtt" #define ENVIAR_SENYAL_NO_DETECTADO "NotSensorMqtt"
Las dos primeras constantes corresponden a los mensajes vía MQTT que la ESP32 tendrá que reconocer para determinar si hay ausencia o presencia de botella. Y las dos últimas constantes corresponden al comando de activación o desactivación del programa del robot

Y por último se utiliza una variable tipo enumerado (estado_sensor) que permite definir los posibles estados del sensor.

6.2.2. Funciones de comunicación

En este caso no hay ninguna función que realice ningún papel adicional solamente el módulo realiza funciones de comunicación MQTT, que actúa como intermediario entre la simulación y la ESP32.

Suscripción al topic

La ESP32 se suscribe al topic "**giirob/pr2/station/mesa/status**". En dicho topic, la ESP32 recibirá los mensajes del sensor simulado en el entorno de RoboDK.

Lectura de mensajes del topic

Cuando se recibe un mensaje de MQTT, la ESP32 analiza dicho contenido, comparándolo con los valores predefinidos:

1. Si el mensaje es “sensor_detecta” el estado de la variable “Estado_lectura_sensor” se actualiza como DETECTADO.
2. Si el mensaje es “sensor_no_detecta”, el estado de la variable “Estado_lectura_sensor”se establece como NO_DETECTADO.
3. En el caso, de que llegue otro mensaje, como es el caso de “online” (mensaje de confirmación de arranque del programa de RoboDK con MQTT) el estado de la variable “Estado_lectura_sensor” se modifica a DESCONOCIDO.

Una vez se haya actualizado la variable se llama a la función de enviar mensaje a MQTT.

La variable “Estado_lectura_sensor” será global, pero no tendrá riesgo de condición de carrera, debido a que no se modifica el estado hasta que se envíe el mensaje después de analizar el estado del sensor. Además,para que se modifique la variable “Estado_lectura_sensor” debe volver a llegar otro mensaje vía MQTT.

Enviar Mensaje Para arrancar el programa:

Una vez se haya analizado el mensaje enviado por sensor simulado en RoboDK y se haya realizado la actualización de la variable “Estado_lectura_sensor”. Se enviará un mensaje al sistema MQTT dependiendo del estado leído anteriormente:

1. Si se detecta botella (Estado_lectura_sensor = DETECTADO), se envía el mensaje de “SensorMqtt” y activando el programa de recogida de botellas del UR3e

2. Si no se detecta una botella (Estado_lectura_sensor = NO_DETECTADO),se envía el mensaje de “NotSensorMqtt”. Provocando que el robot espere hasta que recibe la señal de que haya una botella.
3. En el caso desconocido (Estado_lectura_sensor = DESCONOCIDO), no se hará ninguna acción.

Todos estos mensajes se envían al topic "**giirob/pr2/station/mesa/commands**", cosa que, provocará que RoboDK los analice para determinar las acciones que deberá realizar la simulación

6.2.3 Esquema de comunicación

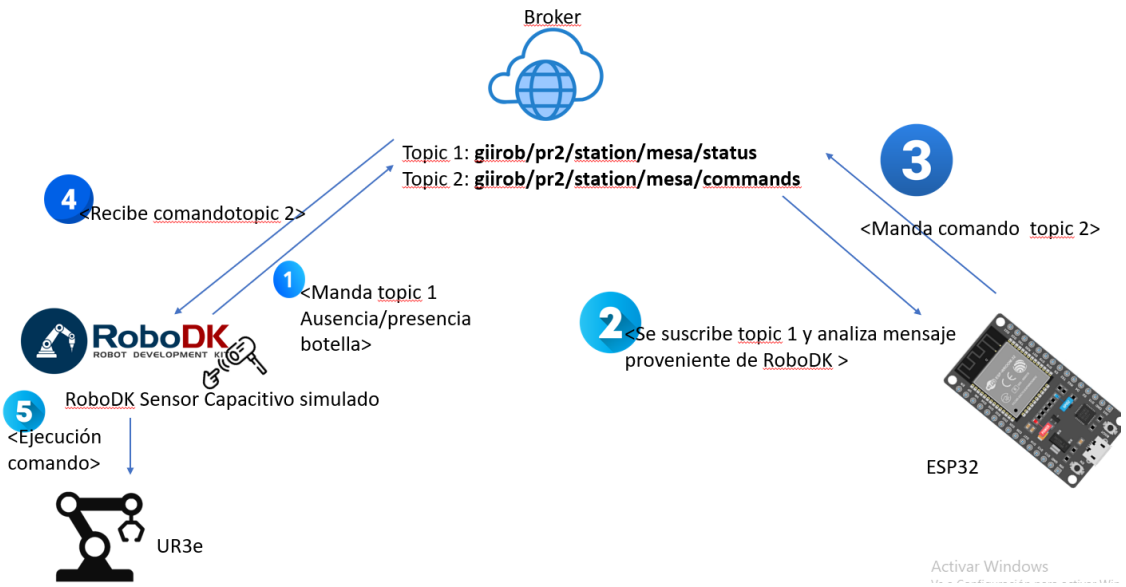

Ilustración 16. Esquema de Comunicación Escenario 2

1. El sensor simulado en RoboDK detecta las botellas en la simulación
2. Dicha señal es enviada al topic **“giirob/pr2/station/mesa/status” y** analizada por la ESP32
3. La ESP32 tras analizar la señal envía un comando al topic **“giirob/pr2/station/mesa/commands”**
4. El comando es analizado por RoboDK
5. RoboDK actúa dependiendo del comando recibido.

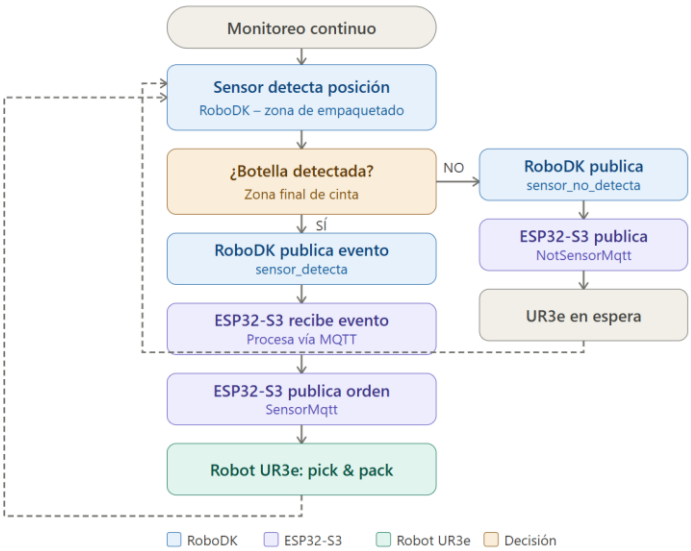

Ilustración 17. Diagrama Flujo Información Sensor RoboDK-ESP32-MQTT

7. Puesta en marcha

7.1 Puesta en marcha RoboDK

Para esta entrega se entregan dos estaciones “DalsyConBD” y “DalsySinBD”, para así permitir comprobar el código sin necesidad de crear una BD

7.1.1 Puesta en marcha Robodk con BD

Para ello, es necesario cumplir previamente una serie de requisitos que permiten la correcta conexión y funcionamiento de la base de datos.

En primer lugar, se debe disponer de pgAdmin y PostgreSQL instalados en el sistema, ya que estas herramientas permiten la gestión y administración de la base de datos relacional utilizada en el proyecto.

Una vez instaladas, se debe ejecutar en PostgreSQL el script CrearBD.sql, incluido en la entrega. Este script es el encargado de crear la estructura inicial de la base de datos, incluyendo las tablas y relaciones necesarias para el funcionamiento del sistema.

Además, el servidor donde se ejecute dicho script debe estar configurado con los siguientes parámetros de conexión:

- Host: 127.0.0.1
- Base de datos: postgres
- Usuario: postgres
- Contraseña: 30082004Aa
- Puerto: 5432

Estas credenciales permiten la conexión local entre la aplicación y el sistema de gestión de bases de datos, garantizando la correcta comunicación con los scripts desarrollados para el control de la producción.

También será necesario disponer de las bibliotecas de PostgreSQL y MQTT instaladas en el entorno Python de RoboDK.

Una vez cumplidos los requisitos anteriores, se procede a la ejecución del sistema. Para ello, se abre el proyecto “DalsyConBD” en RoboDK.

Dentro del entorno, se accede a la carpeta BaseDatos y se abre el script ConsultarBD desde Visual Studio Code, ejecutándolo en primer lugar. Este script se encarga de mantener la supervisión del estado de los lotes en la base de datos.

A continuación, se accede a la carpeta PROGRAMA_PRINCIPAL, donde se encuentra el programa main. Este se ejecuta mediante doble clic y actúa como punto de entrada del sistema, iniciando el resto de programas necesarios para el funcionamiento completo de la simulación.

7.1.2 Puesta en marcha Robodk sin BD

Para poner en marcha RoboDK sin la base de datos, el proceso se simplifica notablemente, siendo únicamente necesario disponer de la librería de **MQTT** instalada en el entorno de Python.

En este caso, se debe abrir el proyecto **"DalsySinBD"**. Dentro del mismo, se accede a la carpeta **PROGRAMA_PRINCIPAL**, donde se encuentra el programa **main**. Este se ejecuta mediante doble clic y actúa como punto de entrada del sistema, iniciando el resto de programas necesarios para el correcto funcionamiento de la simulación.

7.2 Puesta en Marcha Arduino y MQTT

Para arrancar los 2 códigos debemos tener los siguientes requisitos:

1. Tener las siguientes librerías descargadas de Arduino:
    a. ArduinoHttpClient by Arduino | versión 0.6.1
    b. Arduino_DebugUtils by Arduino | versión 1.50
    c. Arduino_ESP32_OTA by Arduino | versión 0.3.1
    d. ArdunioJson by Benoit Blanchon... | versión 7.4.3
    e. LiquidCrystal I2C by Frank de Brabander | versión 1.1.2
    f. PubSubClient by Nick O'Leary | versión 2.8

En el README del repositorio de github también están las librerías

2. En este caso se necesitan 2 ESP32
    a. La primera simplemente con que esté conectada al puerto COM del ordenador. Podrá correr el código del sensor detecta botella dándole a “upload” y funcionará correctamente si se inicia el programa de RoboDK

    b. La segunda es diferente a la primera. Tienes que montar varias cosas, el esquema del termistor (NTC), el esquema del botón de emergencia y la visualización del LCD, y también debe montar el esquema del led Azul. Para que se obtengan temperaturas adecuadas. En el caso de que la temperatura solamente otorga valores muy elevados. Asegurarte que el NTC dé buen contacto o si es necesario por la especificación del NTC que uses, modifica la variable “Temperatura_maxima_permitida” aumentando dicho valor.
    Por otra parte si hay un mal contacto el programa devuelve -1.0ºC indicando que hay un error en el ADC en la LCD. Pero el programa seguirá funcionando correctamente. Si no hay nada conectado a la ESP32 puede darse el caso de devolver valores entre 70-80 ºC.
    Una vez montado el circuito y se hayan hecho ajustes debido al material ya puedes pulsar el botón de “upload” y empezará a medir temperatura. Y cuando aporte calor a la resistencia NTC observará que si se arranca el programa de RoboDK se le enviará un mensaje de temperatura alta y se detendrá la ejecución y cuando presione el botón de emergencia se suspenderán los hilos hasta que se libere la seta

Todos los códigos se encuentran en el repositorio de github y los ficheros Arduino dentro del fichero “Codigos_ino_mqtt”.

Para la conexión MQTT se utiliza la aplicación MQTTX (Color verde) y para suscribirse debe seguir los siguientes pasos:

1. Debe asegurarse que la conexión WIFI esté dentro de la UPV. Si no, modifica el valor de estas variables a una conexión wifi distinta:
    a. NET_SSID y NET_PASSWD
2. Una vez inicializado MQTTX vamos al apartado de “Connections” y le damos a “new connection”.
3. En “new connection” debe introducir los siguientes datos.
    a. Name: Puedes poner el nombre que quieras
    b. Host: mqtt// broker.emqx.io
    c. Port: 1833

   d. Client ID: no modificar
   e. Username: giirob222
   f. Pasword: UPV2024

Se puede modificar esto en las variables MQTT de ambos config.h

4. Cuando ya hemos introducido los datos le damos a conectar
5. Ahora le damos al botón de “New Subscription” y en el apartado de topic introducimos:
   a. “giirob/pr2/station/mesa/status” Y le damos a confirmar
   b. Y vuelves a darle “New Subscription” y escribes: “giirob/pr2/station/mesa/commands” y le das a confirmar

Una vez ya has seguido los 5 pasos asegúrate que MQTTX esté iniciado y ya podrá ver mensajes que se envían por conexión MQTT.

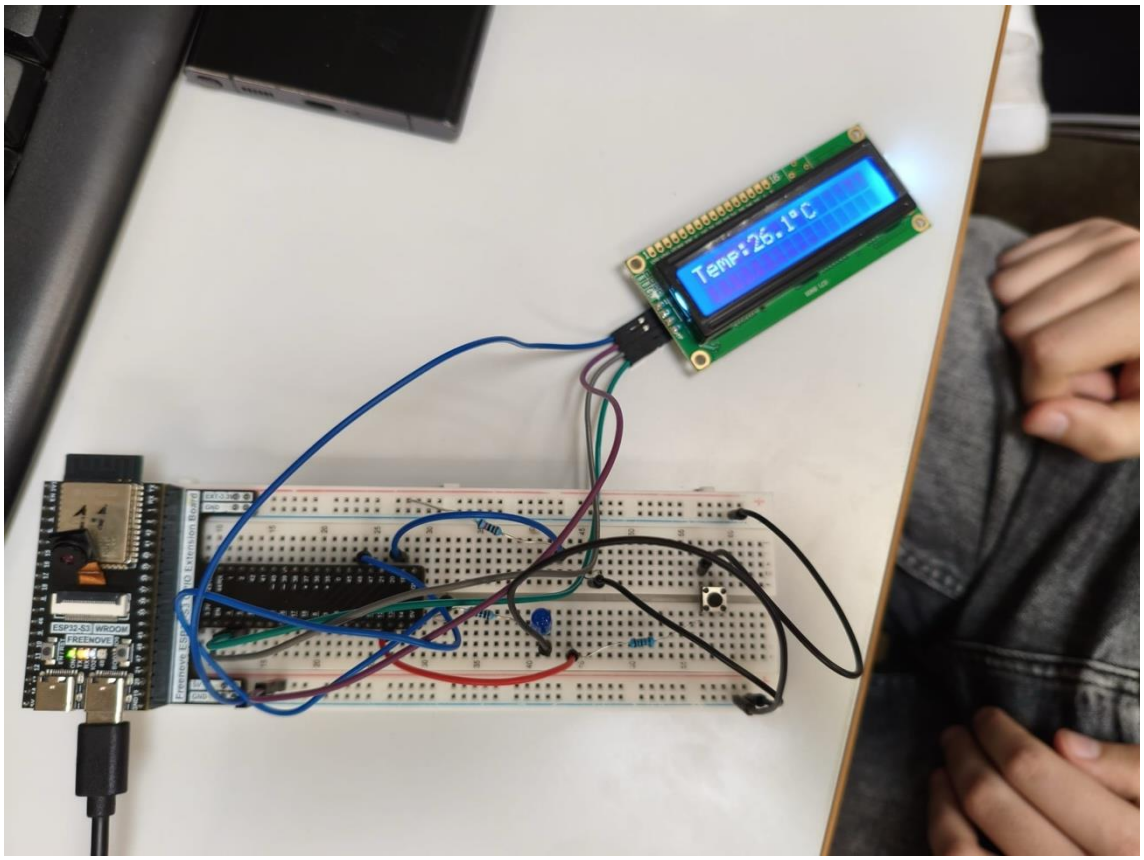

Ilustración 18. Montaje del Circuito de Medición de Temperatura

8. Anexo de Imágenes

Ilustración 1. Diagrama de Flujo Llenado de Botellas Manual 4
Ilustración 2. Diagrama de Flujo Empaquetado y Paletizado Manual 4
Ilustración 3. Celda Llenado y Empaquetado de Botellas Robotizada 6
Ilustración 4. Escenarios de Comunicación de la Celda Robotizada 7
Ilustración 5. Organización en Árbol y Carpetas de la Celda en RoboDK 15
Ilustración 6. Scripts MqttListener y RobotController 18
Ilustración 7. Esquema de Conexión ESP32-Seta Emergencia 24
Ilustración 8. Curva Característica NTC 26
Ilustración 9. Esquema de Conexión NTC 26
Ilustración 10. Tarea Productora 27
Ilustración 11. Tarea Consumidor 27
Ilustración 12. Esquema de Conexión ESP32-LCD 28
Ilustración 13. Esquema de Comunicación Escenario 1 30
Ilustración 14. Diagrama Flujo Información NTC-ESP32-MQTT 31
Ilustración 15. Esquema Conexión Circuito de Medición de Temperatura 31
Ilustración 16. Esquema de Comunicación Escenario 2 34
Ilustración 17. Diagrama Flujo Información Sensor RoboDK-ESP32-MQTT 35
Ilustración 18. Montaje del Circuito de Medición de Temperatura 39